

Git 版本管理与分支合并冲突实验

计算机系统基础 Lab0 实验报告

Yueyi Yang 24300220032

2026 年秋季学期

摘要

为理解 Git 的版本管理机制与分支协作流程，本实验以课程模板仓库中的 `main.c` 为对象，完成代码修改、暂存提交、远程推送和分支合并。在共同提交基础上，分别向 `main` 与 `feature` 分支的相同位置插入不同输出语句，构造内容冲突；通过分析冲突标记、保留双方修改并建立合并提交，完成冲突处理。实验记录表明，初次代码修改已推送至 `GitHub`，本地分支冲突已解决。结合提交信息规范与语义化版本的阅读，进一步分析了暂存区、提交历史及分支机制在代码追溯和协同开发中的作用。

关键词：Git；版本控制；暂存区；分支管理；合并冲突

1 引言与实验环境

版本控制需要记录代码如何变化，并在多人或多条开发线并行工作时保持修改可追踪。本实验依据课程 `GitLab` 任务要求，以一个简单 C 程序为载体，将代码修改、版本保存、远程同步和冲突解决连接为完整流程[1]。实验重点在于理解各操作之间的关系，以及合并冲突发生后如何判断和保留有效修改。

实验使用 Windows 下的 WSL Ubuntu 与 VS Code，远程仓库托管于 `GitHub`。表 1 列出实验环境及主要对象。本文中的 `GitLab` 为课程实验名称。

表 1 实验环境与仓库信息

项目	配置或对象
操作环境	Windows、WSL Ubuntu、Bash
编辑与版本管理	Visual Studio Code、Git、GitHub
编译工具	GCC、Make
模板仓库	ICS-26Fall-FDU/GitLab
个人仓库	yyyang06/ICS-26Fall-FDU-github
实验文件与分支	<code>main.c</code> ； <code>main</code> 、 <code>feature</code>

实验按“修改与提交—分支开发—冲突处理—结果分析”的顺序展开。命令按操作逻辑整理；在 VS Code 中执行的版本管理操作以对应 Git 命令说明。

2 Git 基本原理与问题分析

2.1 协作经历与分工方式

我曾与同学通过 GitBook 协作编写资料。我们按章节分工，每位成员在独立分支完成自己的内容，再由负责人检查并合并到主分支。

这种协作让我体会到，分工需要明确内容边界，版本管理则负责记录修改的作者、时间与内容。独立分支可以隔离尚未完成的工作，合并前的检查能够减少遗漏，也比不断传递不同版本的文件更容易追踪。

2.2 暂存与提交分离的设计目的

工作区保存正在编辑的文件，暂存区组织下一次提交的内容，本地仓库保存提交形成的版本快照。`git add` 将选定内容写入暂存区，`git commit` 再把暂存区的状态记录为一次提交。[3]



图 1 工作区 暂存区与本地仓库的关系

分成两步使一次提交能够围绕一个清晰目的组织。例如，我同时修改 `main.c` 和实验报告时，可以先暂存并提交代码，再单独提交报告；也可以用 `git add -p` 选择同一文件中的部分修改。这样便于审查、定位问题和撤销某一项改动。

暂存还是一次检查机会：`git diff` 查看尚未暂存的修改，`git diff --cached` 查看即将提交的内容。需要注意，暂存保存的是执行 `git add` 时的内容；暂存后再次编辑文件，新的修改仍需重新暂存才能进入下一次普通提交。

2.3 `git branch` 与 `git branch -a` 的区别

表 2 分支查看命令的显示范围

命令	显示范围
<code>git branch</code>	列出本地分支；通常用 <code>*</code> 标记当前分支。
<code>git branch -a</code>	同时列出本地分支和本地保存的远程跟踪分支。
<code>git branch -r</code>	仅列出远程跟踪分支。

其中 `-a` 是 `--all` 的缩写。例如，除了 `main` 和 `feature`，还可能看到 `remotes/origin/main`。远程跟踪分支是本地记录的远程状态，`git branch -a` 本身不会联网刷新；需要更新时先执行 `git fetch`。[2]

3 实验方法与实施过程

3.1 个人仓库建立与克隆

以课程提供的 GitLab 模板为起点建立个人仓库 ICS-26Fall-FDU-github，并在 WSL 中通过 SSH 克隆。进入仓库后，使用 ls 检查文件，可见 main.c、Makefile 和 README.md。后续操作均在该仓库目录内完成。

```
git clone git@github.com:yyyang06/ICS-26Fall-FDU-github.git
cd ICS-26Fall-FDU-github
git status
```

```
yyy@yyy:~/test-git$ git clone git@github.com:yyyang06/ICS-26Fall-FDU-github.git
Cloning into 'ICS-26Fall-FDU-github'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (6/6), done.
Receiving objects: 100% (8/8), done.
remote: Total 8 (delta 0), reused 7 (delta 0), pack-reused 0 (from 0)
```

图 2 个人仓库克隆成功的终端记录

3.2 TODO 修改与首次提交

在已有 Hello, world! 输出之后增加 printf("Hello,ICS!\n");，完成 TODO 要求。图 3 的绿色差异行清楚显示新增内容；图 4 显示推送完成，main 从 47d8d04 更新到 247f714。



```
home > yyy > test-git > ICS-26Fall-FDU-github > C main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("Hello, world!\n");
7 }
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("Hello, world!\n");
7+    printf("Hello,ICS!\n");
8 }
```

图 3 初次提交前后 main.c 的差异

```
git add main.c
git commit -m "add HelloICS"
git push origin main
```

```
• yyy@yyy:~/test-git/ICS-26Fall-FDU-github$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
• yyy@yyy:~/test-git/ICS-26Fall-FDU-github$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 309 bytes | 309.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:yyyang06/ICS-26Fall-FDU-github.git
  47d8d04..247f714  main -> main
○ yyy@yyy:~/test-git/ICS-26Fall-FDU-github$
```

图 4 初次提交后的状态检查与推送记录

3.3 远程提交核对

推送后，在 GitHub 的 main 分支打开 main.c，可以看到新增的 Hello,ICS! 输出语句，以及提交短哈希 247f714。网页中的内容与本地差异一致，说明初次 TODO 修改已经同步到个人远程仓库。

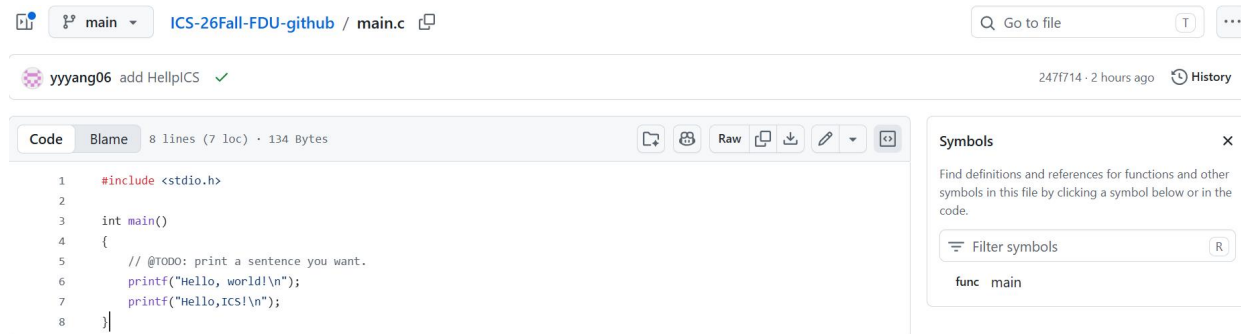


图 5 GitHub 上的 main.c 与初次提交记录

3.4 编译与运行验证

使用 Makefile 编译程序并执行。该次运行发生在创建 feature 分支之后、加入分支专属输出之前，因此终端输出为 Hello, world! 和 Hello,ICS! 两行；随后执行 make clean 清理 main.o 与 main。

```
make
./main
make clean
```

```
• yyy@yyy:~/test-git/ICS-26Fall-FDU-github$ git branch feature
• yyy@yyy:~/test-git/ICS-26Fall-FDU-github$ git switch feature
  Switched to branch 'feature'
• yyy@yyy:~/test-git/ICS-26Fall-FDU-github$ make
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
• yyy@yyy:~/test-git/ICS-26Fall-FDU-github$ ./main
Hello, world!
Hello,ICS!
• yyy@yyy:~/test-git/ICS-26Fall-FDU-github$ make clean
rm -f main.o main
```

图 6 初始修改版本的编译 运行与清理记录

编译过程调用 gcc -Wall -O2，终端记录中未出现编译错误。此次检查验证了初次增加的输出语句可以运行，也避免将生成的可执行文件误当作本次需要提交的源代码改动。

3.5 分支修改与独立提交

在共同版本 247f714 上创建 feature 分支。两个分支均在 Hello,ICS! 语句之后、函数结束之前插入一条新输出：main 输出 Hello,main!，feature 输出 Hello,feature!。二者占据相同插入位置，为后续产生冲突提供条件。

```
git branch feature
git switch main
# 修改 main.c, 加入 main 分支输出
git add main.c
git commit -m "add main"
git switch feature
# 修改 main.c, 加入 feature 分支输出
git add main.c
git commit -m "add feature"
```

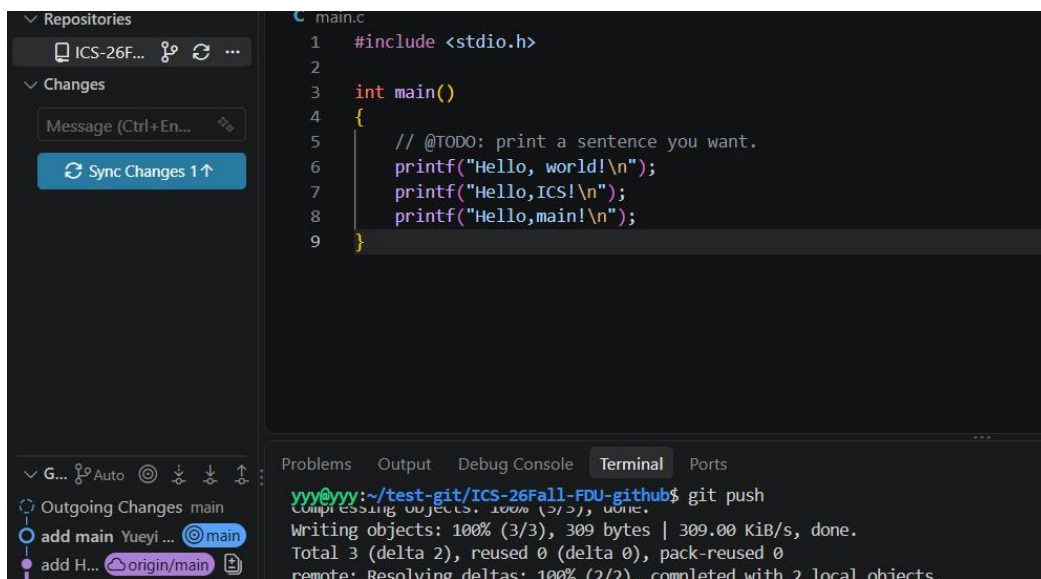


图 7 main 分支修改及 add main 提交

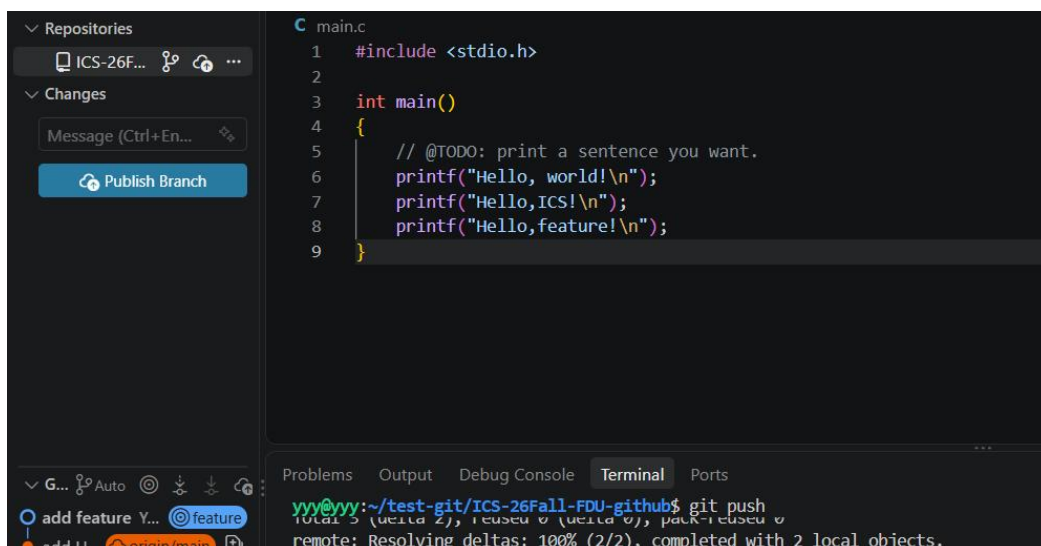


图 8 feature 分支修改及 add feature 提交

切换分支后，工作区展示对应分支的文件内容。feature 从共同版本创建，不会因为后来 main 上新增了提交就自动包含该修改；需要通过合并将两条开发线汇总。

4 合并冲突与实验结果

4.1 冲突构造与定位

```
git switch main
git merge feature
```

Git 尝试自动合并 main.c 后报告内容冲突。两个分支在共同版本的同一位置加入了不同语句，Git 无法可靠决定取舍及排列顺序，因此暂停合并，要求人工处理。

```
yyy@yyy:~/test-git/ICS-26Fall-FDU-github$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
yyy@yyy:~/test-git/ICS-26Fall-FDU-github$
```

图 9 git merge feature 触发内容冲突

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Hello, world!\n");
7      printf("Hello, ICS!\n");
8      <<<<<<< HEAD (Current Change)
9      printf("Hello,main!\n");
10     =====
11     printf("Hello,feature!\n");
12     >>>>>>> feature (Incoming Change)
13 }
```

图 10 main.c 中的双方修改与冲突标记

<<<<<<< HEAD 到 ===== 之间是当前 main 分支的内容；===== 到 >>>>>>> feature 之间是 feature 分支的内容。只修改同一个文件并不必然冲突，关键是两边的改动是否在相同或相互影响的区域无法自动协调。[6]

4.2 冲突消解与合并提交

两条新增输出都符合本次实验目的，因此采用保留双方内容的方案：先输出 Hello,main!，再输出 Hello,feature!，删除全部冲突标记并保存文件。然后暂存解决后的 main.c，完成合并提交。对应命令如下。

```
git add main.c
git commit -m "Merge branch 'feature'"
```

解决冲突不仅是删除标记，还要确认最终内容表达正确意图。本例中两条语句互不排斥，故可以同时保留；如果双方修改涉及互相矛盾的条件或接口，则不能直接接受全部更改。

4.3 合并结果分析

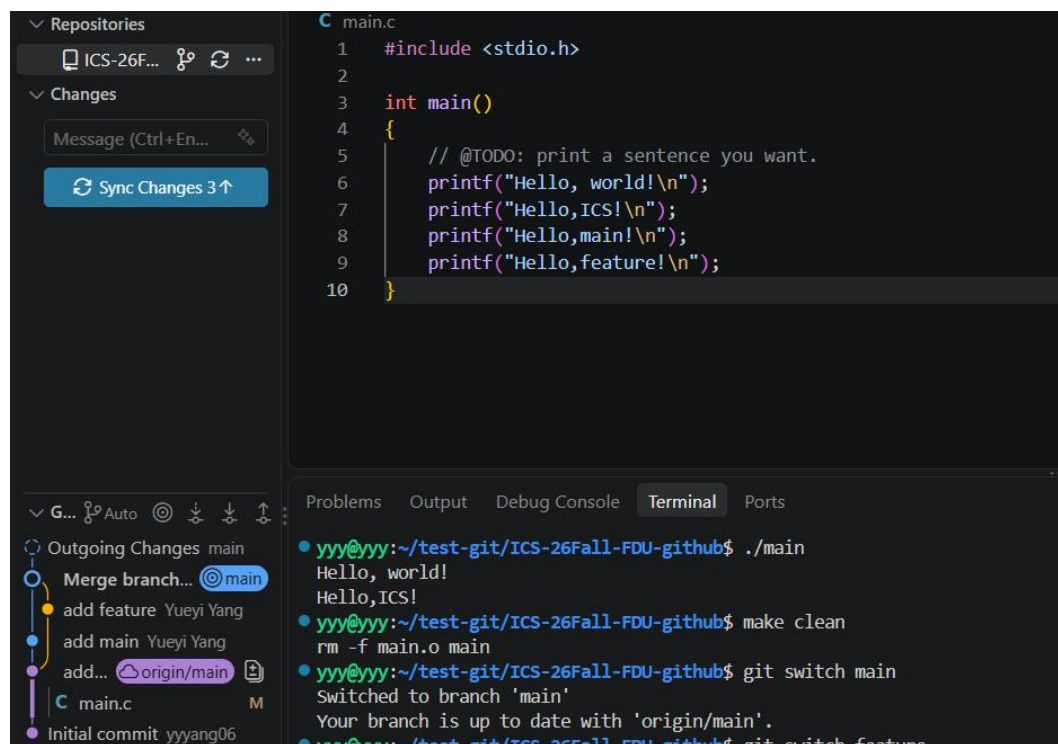


图 11 冲突解决后的代码及 main 分支合并记录

图 11 中冲突标记已消失，main.c 同时保留两条分支输出，左侧历史图出现 Merge branch 合并节点，当前分支为 main。由此可以确认，本地冲突处理和合并提交已经完成。

4.4 最终源代码

```
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("Hello, world!\n");
    printf("Hello, ICS!\n");
    printf("Hello, main!\n");
    printf("Hello, feature!\n");
}
```

最终代码预期依次输出 Hello, world!、Hello, ICS!、Hello, main! 和 Hello, feature!。图 11 的终端仍保留合并前的历史输出，不能作为最终版本已经重新运行的证明。

5 拓展阅读与讨论

5.1 提交信息规范

《Commit message 和 Change log 编写指南》介绍了一种结构化提交信息约定：标题用类型、可选范围和简述表达改动，正文说明动机及行为变化，页脚可记录不兼容变更或关联 Issue。统一格式有助于检索历史和生成变更日志。[4]

结合本实验，我认为提交信息应让读者不打开差异也能理解修改目的。例如，完成 TODO 可写为 feat(main): add ICS greeting，提交报告可写为 docs: add GitLab report。这是对后续写法的改进建议，实验截图中的既有提交信息保持原样。

5.2 语义化版本

《语义化版本 2.0.0》以公共 API 为判断基础，用“主版本号.次版本号.修订号”传达兼容性变化。它帮助使用者判断升级风险，减少依赖管理中的歧义；已经发布的版本不应直接覆盖修改，而应发布新版本。[5]

表 3 语义化版本的递增规则

变化类型	版本递增规则	示例
不兼容的 API 修改	主版本号增加，后两位归零	1.2.3 → 2.0.0
向下兼容的功能新增	次版本号增加，修订号归零	1.2.3 → 1.3.0
向下兼容的问题修复	修订号增加	1.2.3 → 1.2.4

提交与发布的粒度不同：一次提交记录开发过程中的一组修改，版本号则说明一个发布版本的兼容性含义。因此，不能简单地把每个 commit 都视为一次主版本或次版本升级。

5.3 学习 Git 的意义

首先，Git 让修改过程可追溯。本实验中，初次增加 ICS 输出、main 分支修改、feature 分支修改和最终合并各自对应独立记录。出现问题时，可以比较不同版本，判断问题由哪次修改引入，而不必依赖文件名中的“最终版”等标记。

其次，分支支持并行尝试。不同成员可以从共同版本出发分别工作，随后合并成果。本实验的冲突说明，Git 能定位内容分歧，但无法替开发者决定业务意图；解决冲突时仍需要理解双方修改并检查最终程序。

最后，Git 提供了可重复的工作流程。将“检查改动—测试—暂存—提交—推送”作为习惯，可以明确区分本地保存与远程共享。规范的提交信息解释每次修改，语义化版本解释每次发布，两者共同提高协作和维护效率。

6 结论

实验完成了 main.c 的初次修改及远程推送，并在 main 与 feature 分支之间构造和解决了内容冲突。结果表明，分支能够隔离独立修改，合并机制能够识别自动整合无法处理的分歧；最终内容的选择仍需开发者根据修改目的作出判断。

参考文献

- [1] 课程实验说明与模板[EB/OL]. <https://ics-26fall-fdu.github.io/labs/lab0-git-lab/> <https://github.com/ICS-26Fall-FDU/GitLab>
- [2] Git 官方文档 git branch[EB/OL]. <https://git-scm.com/docs/git-branch>
- [3] Git 官方文档 git add[EB/OL]. <https://git-scm.com/docs/git-add>
- [4] 阮一峰 Commit message 和 Change log 编写指南[EB/OL]. https://www.ruanyifeng.com/blog/2016/01/commit_message_change_log.html
- [5] 语义化版本 2.0.0[EB/OL]. <https://semver.org/lang/zh-CN/>
- [6] Git 官方文档 git merge[EB/OL]. <https://git-scm.com/docs/git-merge>